Shortest-Path Planning in a Large-Scale IoV Mobility Network

Dijkstra and A* on the DIMACS Colorado road network (USA-road-d.COL)

Table of Contents

Summary

This report studies shortest-path planning on a real road network (Colorado, 435,666 intersections, 521,200 roads) used as a model of vehicular (IoV) mobility. I implemented Dijkstra's algorithm (binary min-heap) and A* (with a straight-line and a landmark heuristic) from scratch, split the network into disjoint subgraphs of about 1,000 to 52,000 vertices, and compared them on 15 random source–destination pairs per size. All algorithms returned identical optimal costs; A* explored 2–17× fewer vertices and ran up to about 5× faster at 50k vertices. A bonus experiment with dynamic edge costs $w = \alpha \cdot d + \beta \cdot c$ shows that even mild congestion changes the best route in most queries, and that precomputed heuristics must be refreshed if costs decrease.

A. Dataset

- **Name:** USA-road-d.COL (Colorado road network), 9th DIMACS Implementation Challenge: Shortest Paths. Files used: `USA-road-d.COL.gr` (arcs) and `USA-road-d.COL.co` (coordinates).
- **Source:** http://www.dis.uniroma1.it/~challenge9 (derived from US Census Bureau TIGER/Line).
- **Description:** real road network used as a standard routing benchmark.
- **Size:** 435,666 nodes; 1,057,066 arc records (14,666 parallel duplicates collapsed → 1,042,400 arcs = 521,200 two-way road segments).
- **Conversion:** vertex = road intersection/endpoint with (longitude, latitude); directed edge = drivable road segment (each road stored in both directions); weight = segment length in the dataset's distance units (≈ 10 units per metre; measured min 7.03, mean 9.99 units/m against great-circle length). Only `p`, `a` (graph) and `v` (coordinates) lines are used.
- **Why suitable:** real topology (sparse, near-planar), large enough for 1k–435k scaling, positive weights, coordinates available for a geometric heuristic, and it models vehicular (IoV) route planning directly.

B. Graph construction

Adjacency list `adj[u] = [(v, w), ...]`. `preprocess.py` partitions the graph into **disjoint, connected** subgraphs by BFS region growing (next seed from the previous frontier; regions below half the target are merged into their smallest neighbour). Per size it writes

`vertices_N.csv`, `edges_N.csv`, and one `stats.csv`. Arcs between different subgraphs are dropped (counted as cut arcs).

Full graph: 435,666 vertices, 1,042,400 arcs (521,200 undirected), average degree 2.39, maximum degree 8.

Target	# subgraphs	Vertices min / mean / max	Avg degree	Max degree	Cut arcs
1,000	379	505 / 1,149 / 1,713	2.35	8	18,246
5,000	79	2,561 / 5,514 / 7,667	2.37	8	8,788
10,000	40	5,601 / 10,891 / 15,445	2.38	8	6,088
25,000	17	14,517 / 25,627 / 27,419	2.38	8	3,672
50,000	8	52,068 / 54,458 / 63,205	2.39	8	3,058

Every subgraph is a single connected component (verified), and every vertex belongs to exactly one subgraph. Sizes are approximate (about 0.5×–1.5× the target); per-subgraph numbers are in `stats.csv`.

C. Dijkstra

Binary-heap (`heapq`) Dijkstra with lazy deletion and early exit when the destination is settled; no graph library (scipy was used only as an independent check on 10 full-graph queries: all matched). *Vertices explored* = vertices settled; *edges examined* = edge relaxations attempted.

Complexity: every vertex is settled once, every edge scanned once, and each successful relaxation pushes one heap entry (≤ E pushes of cost O(log V)): **O((V + E) log V)**, i.e. O(V log V) here since E = O(V). Space O(V + E).

D. A*

`f(v) = g(v) + h(v)`. Two heuristics were implemented:

1. **Straight-line (coordinates):** $h(v) = c \cdot \|pos(v) - pos(t)\|$, with positions projected to metres and c = the minimum of w(u,v)/length(u,v) over the subgraph's edges. Every edge then satisfies w ≥ c·length, so by the triangle inequality any path costs at least $c \cdot \|pos(v) - pos(t)\|$ → **admissible** (and consistent). Because c is a worst-case ratio, the heuristic underestimates on most roads, which limits its pruning.

2. **ALT landmarks:** 8 landmarks by farthest-point selection with exact distances precomputed; h(v) = max_L |d(L,t) − d(L,v)|, admissible and consistent on this symmetric graph by the triangle inequality. Costs a one-off preprocessing step per graph.

Both heuristics are consistent, so each vertex is settled at most once and the first pop of the destination is optimal.

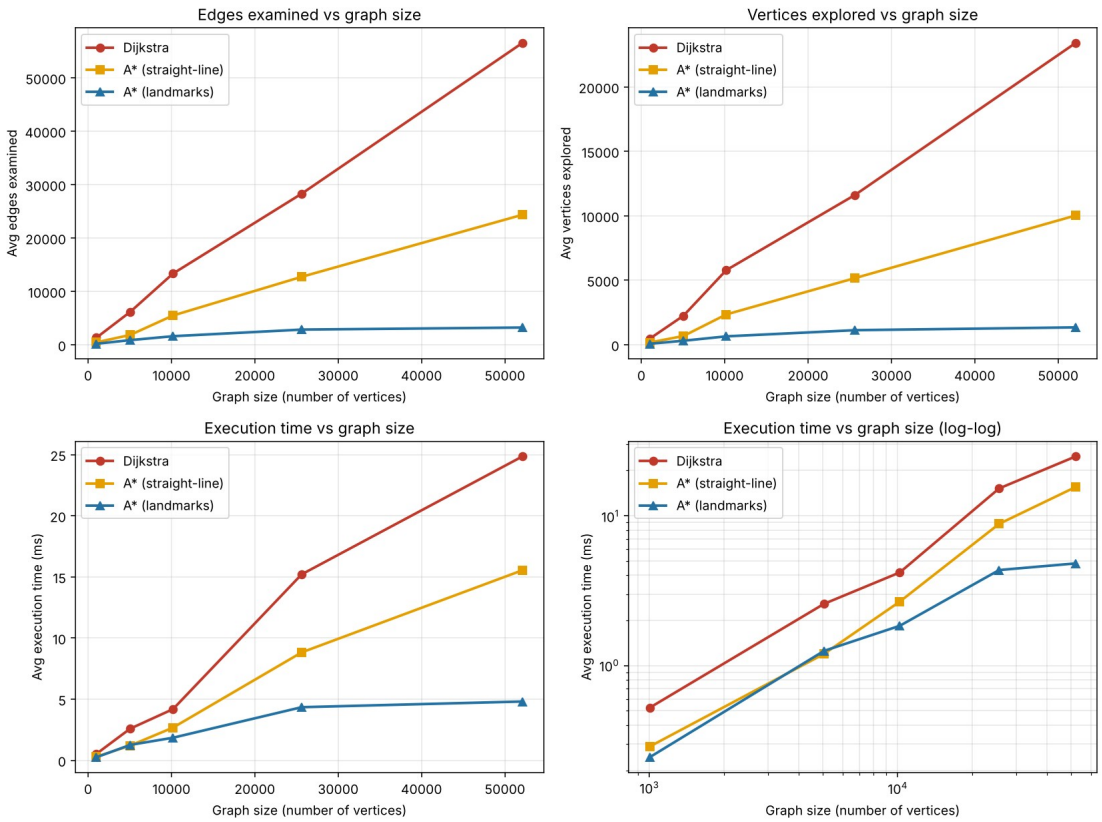
E. Experiments

Subgraphs from `preprocess.py`; for each target size the 3 subgraphs closest to it, 5 random source–destination pairs each (15 queries per size, same pairs for all algorithms). Costs of all three algorithms were asserted equal on every query.

Subgraph size	Algorithm	Queries	Avg cost	Avg vertices explored	Avg edges examined	Avg time (ms)	Heuristic prep (s)
1,000 (avg V = 1,002)	Dijkstra	15	53,642	457	1,315	0.52	0.00
1,000 (avg V = 1,002)	A*-Euclid	15	53,642	162	451	0.29	0.00
1,000 (avg V = 1,002)	A*-ALT8	15	53,642	66	181	0.24	0.01
5,000 (avg V = 5,048)	Dijkstra	15	496,582	2,239	6,141	2.58	0.00
5,000 (avg V = 5,048)	A*-Euclid	15	496,582	667	1,828	1.20	0.00
5,000 (avg V = 5,048)	A*-ALT8	15	496,582	300	861	1.25	0.05
10,000 (avg V = 10,191)	Dijkstra	15	477,280	5,780	13,329	4.18	0.00
10,000 (avg V = 10,191)	A*-Euclid	15	477,280	2,343	5,476	2.67	0.00
10,000 (avg V = 10,191)	A*-ALT8	15	477,280	642	1,597	1.84	0.06

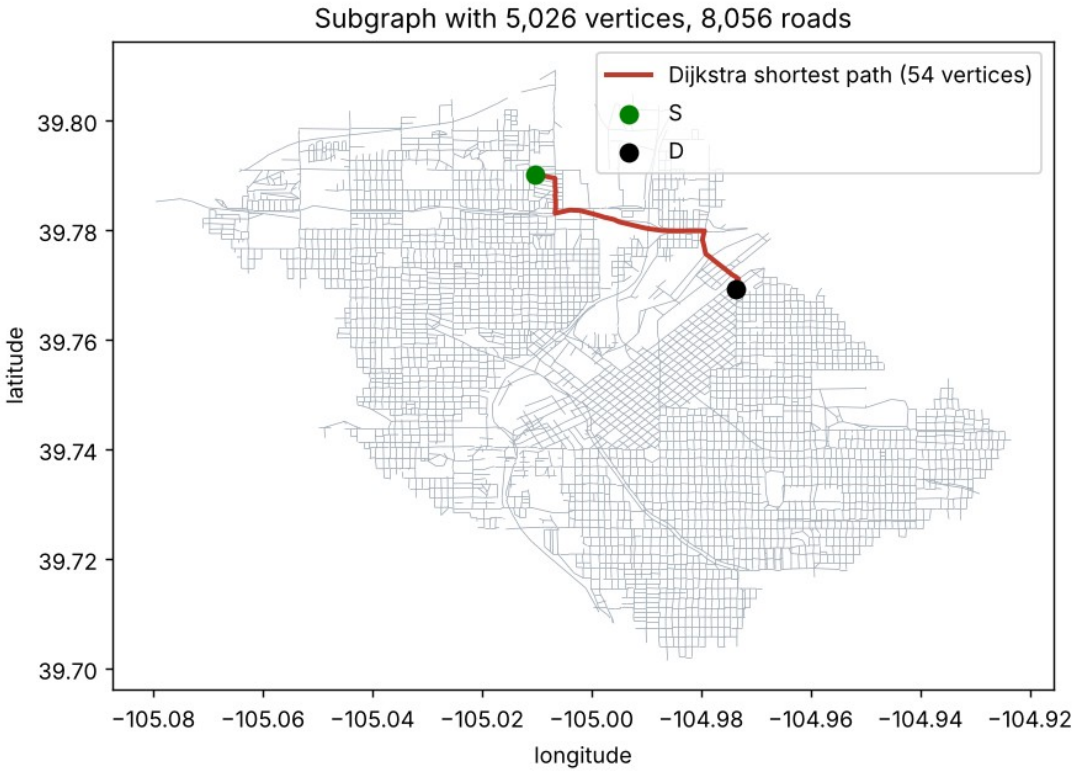
Subgraph size	Algorithm	Queries	Avg cost	Avg vertices explored	Avg edges examined	Avg time (ms)	Heuristic prep (s)
25,000 (avg V = 25,624)	Dijkstra	15	1,049,126	11,614	28,294	15.20	0.00
25,000 (avg V = 25,624)	A*-Euclid	15	1,049,126	5,162	12,722	8.82	0.02
25,000 (avg V = 25,624)	A*-ALT8	15	1,049,126	1,121	2,835	4.34	0.29
50,000 (avg V = 52,105)	Dijkstra	15	860,620	23,415	56,533	24.85	0.00
50,000 (avg V = 52,105)	A*-Euclid	15	860,620	10,024	24,347	15.54	0.03
50,000 (avg V = 52,105)	A*-ALT8	15	860,620	1,338	3,220	4.80	0.54

Figure 1 – Dijkstra vs A*: edges examined, vertices explored, execution time (linear and log-log) against graph size.



Comparison of Dijkstra and A against graph size*

Figure 2 – Example subgraph (5,026 vertices) with a Dijkstra shortest route from S (green) to D (black).



Subgraph map

Observations

- All algorithms return identical optimal costs, confirming admissibility in practice.
- **Straight-line A*** explores about 2.3–3.4× fewer vertices than Dijkstra and runs 1.6–2.2× faster.
- **ALT A*** explores about 7–17× fewer vertices and runs 2.2–5.2× faster from 10,000 vertices up; its advantage grows with graph size (at ~52k vertices: 1,338 vs 23,415 vertices explored). At 1,000–5,000 vertices the time gain is small because the per-expansion heuristic cost (8 landmark lookups in Python) offsets the saving; there the straight-line version is comparable.
- Dijkstra's explored set grows roughly linearly with size; A* with ALT stays focused near the route.

- Trade-off: ALT needs 0.01–0.54 s preprocessing and 8·|V| stored distances per graph, which pays off after a few queries on a fixed graph but must be recomputed if edge costs change.
- A first experiment on nested BFS balls up to the full 435,666-vertex graph (an earlier run on nested subgraphs, landmark heuristic only) gave ≈428 ms for Dijkstra vs ≈160 ms for ALT A* on the full graph.

Limitations

Only 15 queries per size and different subgraphs per size, so average costs are not comparable across sizes and ratios are noisy. Timings are single-run CPython on a shared container. Subgraph sizes are approximate.

F. Analysis questions

(Numbers refer to the disjoint-subgraph experiments in Section E: 15 queries per size; and to the bonus experiments.)

1. Why is the dataset suitable for shortest-path analysis? USA-road-d.COL is the real road network of Colorado from the 9th DIMACS Shortest Paths Challenge (US Census TIGER/Line): 435,666 intersections and 521,200 two-way road segments. It is naturally a weighted graph (intersections = vertices, road segments = edges, segment length = positive weight), so no artificial modelling is needed. It is large enough to scale from 1,000 to about 50,000+ vertices (and the full 435k), it has real road topology (sparse, near-planar, irregular), it comes with node coordinates (needed for a geometric A* heuristic), and it is a standard routing benchmark, which makes results comparable. It models vehicular (IoV) route planning directly.

2. How was the raw dataset transformed into a weighted graph? The `.gr` file lists directed arcs (`a u v w`); the `.co` file lists coordinates (`v id lon lat`, degrees × 10^6). I read only these lines, converted 1-based ids to 0-based, collapsed 14,666 parallel duplicate arcs by keeping the minimum weight (1,057,066 records → 1,042,400 arcs = 521,200 undirected roads), dropped self-loops, and stored an adjacency list `adj[u] = [(v, w), ...]`. Each vertex keeps its (lon, lat). `preprocess.py` then partitions the graph into disjoint connected subgraphs of about 1k, 5k, 10k, 25k and 50k vertices by BFS region growing (small leftover pockets merged into a neighbour), relabelling vertices locally and exporting a vertices CSV and an edges CSV per size, plus `stats.csv`.

3. Do Dijkstra and A* give the same optimal cost? Yes. In every query of every experiment (Part E and the bonus) the code asserts that Dijkstra, A* with the straight-line heuristic and A* with landmarks return exactly the same cost; none failed. This is expected: both heuristics are admissible and consistent (see Q8), so A* is guaranteed optimal. If several shortest paths tie, A* may return a different path with the same cost, which is still optimal. An independent check with scipy on 10 full-graph queries also matched my Dijkstra.

4. Which explores fewer vertices, and why? A*. Averaged per size, straight-line A* explored 2.3–3.4× fewer vertices than Dijkstra, and landmark A* 7–17× fewer (e.g. about 52k-vertex subgraphs: 23,415 for Dijkstra, 10,024 straight-line, 1,338 landmark). Dijkstra orders vertices

only by distance from the source, so it expands a disc of all vertices closer than the destination in every direction. A* orders by g + h, where h estimates the remaining cost, so vertices that lead away from the destination get a large f and are never expanded. The tighter the heuristic is to the true remaining distance, the fewer vertices are explored: landmarks (exact distances to 8 landmarks) are far tighter than the straight-line bound, which has to use the worst road ratio (min weight per metre, about 7.0 versus an average of 10.0) to stay admissible.

5. How does execution time vary with graph size? It grows roughly linearly with the number of vertices for Dijkstra: average query time 0.52 ms (1k), 2.6 (5k), 4.2 (10k), 15.2 (25k), 24.9 ms (~52k); about 428 ms on the full 435k graph in the earlier run. This is consistent with $O((V+E) \log V)$ with $E \approx 1.2V$. A* grows more slowly: straight-line 0.29 → 15.5 ms, landmark 0.24 → 4.8 ms (about 5× faster than Dijkstra at 50k), because it only expands vertices near the route. Time per query also depends on how far apart S and D are, so with 15 random pairs per size the curve is noisy (single-run Python timings). Landmark A* additionally pays a one-off preprocessing cost (0.01 s at 1k to 0.54 s at 50k).

6. Effect of graph density Time is governed by V and E: every scanned edge costs a relaxation and possibly a heap operation, so cost grows with the number of edges examined. My dataset is very sparse (average degree 2.39, maximum 8, $E \approx 1.2 \cdot V$), so I did not vary density experimentally; with these graphs the edges examined per vertex explored stays at about 2.3–2.5 in every experiment (e.g. 13,329 edges for 5,780 vertices), i.e. the work per vertex is proportional to its degree. Theory: with a binary heap Dijkstra is $O((V+E) \log V)$ – near $O(V \log V)$ for road networks, but up to $O(V^2 \log V)$ for a dense graph ($E \approx V^2$), where an array-based $O(V^2)$ implementation or a Fibonacci heap ($O(E + V \log V)$) becomes preferable. Density also reduces how much A* can help: in a sparse near-planar road graph the shortest path is a narrow corridor that a good heuristic can focus on, whereas in a dense graph many near-equal alternatives exist and pruning is weaker.

7. Time complexity of my Dijkstra $O((V + E) \log V)$ with a binary min-heap (Python `heapq`) and lazy deletion: each vertex is settled once, each edge is scanned once, and each successful relaxation pushes one heap entry (at most E pushes, each $O(\log E) = O(\log V)$). Since $E = O(V)$ for road networks, this is $O(V \log V)$. Space is $O(V + E)$. The early exit when the destination is settled does not change the worst case, but reduces the average work.

8. A* heuristics – admissible and consistent? Two heuristics were used. - *Straight-line:* $h(v) = c \cdot \|pos(v) - pos(D)\|$ where positions are projected to metres and c = min over all edges of $w(u,v) / length(u,v)$ (about 7.0 weight units per metre on this data). *Admissible:* every edge has $w \geq c \cdot length$, so any path costs at least $c \cdot$(sum of its segment lengths) $\geq c \cdot$(straight-line distance) by the triangle inequality, hence h never overestimates. *Consistent:* $h(u) = c\|u - D\| \leq c\|u - v\| + c\|v - D\| \leq w(u,v) + h(v)$. Using the minimum ratio is what makes it safe; a larger scale would be tighter but could overestimate and make A* return non-optimal paths. - *ALT landmarks (8 landmarks):* $h(v) = \max$ over landmarks L of $|d(L,D) - d(L,v)|$. *Admissible* by the triangle inequality on the (symmetric) graph, $|d(L,D) - d(L,v)| \leq d(v,D)$; *consistent* because the same inequality gives $h(u) - h(v) \leq d(u,v) \leq w(u,v)$. Consistency means each vertex is settled at most once and the first time D is popped its cost is optimal. Both claims are supported empirically (zero cost mismatches). Caveat from the bonus: the guarantees hold only for the weights the heuristic was built on; if edge costs increase they remain valid, but if any cost decreases a stale

heuristic can become inadmissible (my test: stale landmarks gave a non-optimal path in 18 of 20 queries) and must be recomputed.

9. Difficulties converting the real data into a graph - Parallel/duplicate arcs (14,666) and the need to decide a rule (keep the minimum). - Roads stored as two directed arcs, so the graph had to be treated as symmetric (verified: 0 asymmetric arcs) – important for the landmark heuristic. - The coordinates are in a separate file and in degrees × 10^6; I first only had the `.gr` file and could not download the `.co` file, so I initially used a graph-based heuristic, and later projected lon/lat to metres. - Edge weights are in unusual units (about 10 per metre) and heuristic scaling must be derived from the data, not assumed. - Extracting subgraphs: they must stay connected, vertices must be relabelled consistently, arcs crossing between subgraphs are lost (counted as cut arcs), and exact sizes are hard to hit (final sizes are about 0.5–1.5× the target). - Scale: 1M+ arcs in pure Python needs careful memory use and loading time, and timings are noisy. - Integer rounding of dynamic costs and keeping them symmetric in the bonus experiment.

10. Recommendation for large-scale IoV/UAV route planning Use **A*** over Dijkstra. In my experiments it returned identical optimal costs while exploring 2–17× fewer vertices and running up to about 5× faster, with the gap widening as the network grows, and it needs coordinates, which vehicles and UAVs naturally have. Specific choice: - If the network is mostly static or changes slowly (a fixed road map with many queries): **A* with landmarks (ALT)** – best results here (e.g. 4.8 ms vs 24.9 ms at ~52k vertices; 1,338 vs 23,415 vertices explored); the one-off preprocessing (≤ 0.5 s here) is amortised over many queries. - If costs change frequently (congestion, weather, link quality): **A* with the straight-line heuristic** built on a safe lower-bound scale (it stays admissible for cost increases and needs no precomputation), or recompute landmarks periodically; Dijkstra remains the simplest always-correct fallback, but my bonus runs show that routes must be recomputed because even mild congestion changed the best route in most queries. - For UAV routing in free space, the straight-line (Euclidean/3-D) heuristic is natural and cheap. For very large static road networks, speed-up techniques such as contraction hierarchies would be the next step, but those were outside this assignment.

Bonus – Dynamic edge costs

Model

$w(u,v) = \alpha \cdot d(u,v) + \beta \cdot c(u,v)$

- **d(u,v)** = original road length from the dataset (the weight used in Parts A–E).
- **c(u,v)** = dynamic delay on the road, in the same units as d: $c = \rho \cdot d$, with a congestion factor $\rho \geq 0$ on affected roads and $\rho = 0$ elsewhere. With $\alpha = 1$, a road with $\rho = 2$ costs $(1 + 2\beta) \cdot d$, so at $\beta = 1$ it is 3× as expensive.
- **$\alpha = 1$, $\beta \in \{0, 0.5, 1, 2, 4\}$**. $\beta = 0$ is the original network, whose shortest route is the reference. Weights are rounded to integers (≥ 1) and changed symmetrically in both directions of a road.
- **Scenarios** (which roads get $\rho > 0$):
  - *random*: 20% of roads, $\rho \sim U(1, 3)$ — diffuse congestion.

  - *incident*: all roads inside a disc centred on the middle of the original route (radius 15% of the S–D straight-line distance, at least 300 m), $\rho \sim U(2, 4)$ — a localised jam, accident or weather event on the route.
- **Procedure:** for each subgraph (≈10,000 and ≈25,000 vertices), 20 source–destination pairs whose original route has ≥ 25 vertices. Modify the weights, recompute the shortest path (Dijkstra; A* with straight-line and landmark heuristics for cross-checking), and compare with the original route. All three algorithms returned the same cost in every run (asserted in code). Because $\alpha = 1$ and $\beta \cdot c \geq 0$, new weights are never below d, so heuristics built on d remain admissible.

Script `dynamic_costs.py`; raw numbers in `results_dynamic_costs.csv`; example in `route_change_example.png/.txt`.

Results (averages over 20 queries)

≈25,000-vertex subgraph

Scenario	β	Route changed	Old route's cost under new weights	New optimal cost	Saving by rerouting	Distance detour	Edges shared with old route
random	0	0%	1,451,962	1,451,962	0.0%	+0.0%	100%
random	0.5	100%	1,760,394	1,649,806	6.3%	+3.0%	54%
random	1	100%	2,068,825	1,776,119	14.1%	+7.2%	37%
random	2	100%	2,685,689	1,955,329	27.2%	+14.3%	30%
random	4	100%	3,919,415	2,208,482	43.7%	+21.9%	24%
incident	0	0%	1,451,962	1,451,962	0.0%	+0.0%	100%
incident	0.5	95%	1,990,670	1,613,818	18.9%	+8.0%	37%
incident	1	100%	2,529,375	1,650,897	34.7%	+9.1%	30%
incident	2	100%	3,606,788	1,718,089	52.4%	+9.3%	30%
incident	4	100%	5,761,613	1,843,088	68.0%	+10.4%	26%

≈10,000-vertex subgraph

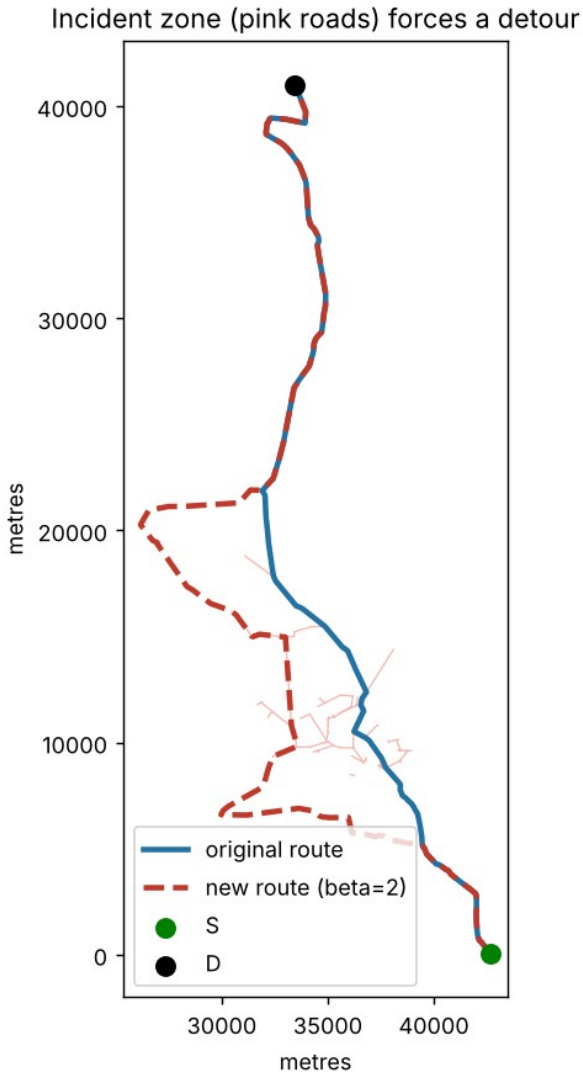
Scenario	β	Route changed	Old route's cost under new weights	New optimal cost	Saving by rerouting	Distance detour	Edges shared with old route
random	0	0%	701,469	701,469	0.0%	+0.0%	100%
random	0.5	70%	845,040	832,073	1.5%	+1.3%	84%

Scenario	β	Route changed	Old route's cost under new weights	New optimal cost	Saving by rerouting	Distance detour	Edges shared with old route
random	1	85%	988,610	945,391	4.4%	+3.8%	80%
random	2	100%	1,275,750	1,147,749	10.0%	+7.5%	72%
random	4	100%	1,850,032	1,527,807	17.4%	+14.7%	68%
incident	0	0%	701,469	701,469	0.0%	+0.0%	100%
incident	0.5	75%	945,520	864,789	8.5%	+11.6%	53%
incident	1	100%	1,189,571	912,355	23.3%	+24.7%	32%
incident	2	100%	1,677,673	942,714	43.8%	+25.9%	31%
incident	4	100%	2,653,877	969,273	63.5%	+38.0%	26%

"Saving by rerouting" = how much cheaper the new optimal route is than simply keeping the old route under the new conditions. "Distance detour" = change in physical length (sum of d) of the new route versus the old one.

Example (10k subgraph, incident, β = 2)

old route: 149 vertices, distance 493876, cost under new weights 1374558 new route: 170 vertices, distance 657102, cost under new weights 1178108



Route change

The old route (blue) crosses the incident zone (pink roads). The new route (red dashed) shares both ends with the old one but takes a west detour around the zone: 170 instead of 149 vertices and ~33% more physical distance (657,102 vs 493,876), yet 14% cheaper under the new conditions (1,178,108 vs 1,374,558).

Discussion

1. **Route choice is sensitive to conditions.** Even mild congestion ($\beta = 0.5$) changed the route for 70–100% of queries in the random scenario and 75–95% in the incident scenario; by $\beta \geq 1$–2 almost every route changed. The shortest path in distance is generally not the best path in a changed network.
2. **Rerouting pays off, more so as congestion grows.** On the 25k subgraph, incident savings rise from 19% ($\beta = 0.5$) to 68% ($\beta = 4$); random-congestion savings from 6% to 44%. Keeping the old route would have cost much more than the new optimum.

3. **Cheaper does not mean shorter.** The new routes are physically longer (detour +3% to +38%) because the optimiser trades distance for avoiding congested roads. Higher β makes delay matter more relative to distance, so detours grow; α/β is effectively the driver's preference between distance and delay.
4. **Localised vs diffuse congestion.** An incident on the route forces a large structural change (only 26–53% of the old edges remain on the 10k graph, 26–37% on the 25k graph) but a detour around a zone can be moderate. Diffuse congestion changes many small segments: overlap drops gradually, and on the larger graph more alternative corridors exist, so more of the route changes.
5. **Effect on the algorithms.** Dijkstra's effort barely changes (≈4.7k → 5.5k explored on 10k; ≈11.9k → 12.4k on 25k) because it explores by cost regardless. A* loses some pruning: weights increase while the heuristic stays based on original distances, so it is a weaker lower bound. On the 25k graph, straight-line A* explores 4,198 vertices at $\beta = 0$ and 4,636–7,291 under congestion; landmark A* goes from 1,096 to 2,430–5,531, but both remain well below Dijkstra's ~12k and stay optimal. Details on stale versus refreshed heuristics, including what happens if costs decrease, are in Appendix A.
6. **Practical implication.** For a mobility/IoV system, routes should be recomputed when conditions change; a static shortest route can be far from optimal. Dijkstra is robust to any change; A* stays correct for cost increases (lower bounds still valid) but needs its heuristic refreshed for decreases.

Limitations

Synthetic congestion (random or a disc), a single subgraph per size, 20 queries per setting and one scenario of each type, so the exact percentages are indicative. The cost model assumes c is proportional to d; real delay can depend on traffic volume or time of day.

Conclusion

The Colorado road network is a suitable large, real, sparse benchmark. Dijkstra is simple and always correct, with $O((V+E) \log V)$ cost; A* returns the same optimal costs while examining far fewer vertices, especially with the landmark heuristic, and its advantage grows with graph size. For large IoV/UAV networks I recommend A* (landmarks for static maps, a safe straight-line bound when costs change often) and recomputing routes as conditions change.

Appendix A – Stale heuristics under changing costs (optional)

Setup (my interpretation of the bonus; adjust if your brief differs). Weights change after heuristics were precomputed. On the ~10k- and ~25k-vertex subgraphs, a random share of roads (both directions, symmetric) gets a multiplier: *increase-light* (20% of roads ×1.5–4), *increase-heavy* (50% ×2–6), *mixed-with-decreases* (30% ×0.3–3). 20 random queries per case, same pairs for every variant. Dijkstra on the new weights is the reference; "stale" heuristics were built on the original weights, "refreshed" ones are rebuilt on the new weights. Script: `dynamic.py`; raw numbers: `results_dynamic.csv`.

Theory. If costs only increase, d_new ≥ d_old, so a stale h ≤ d_old ≤ d_new is still admissible and consistent: A* stays optimal. If any cost decreases, stale h can exceed the true remaining cost, so A* may return a non-optimal path.

Results (≈25k-vertex subgraph; averages over 20 queries)

Scenario	Algorithm	Vertices explored	Time (ms)	Non-optimal paths	Refresh time (s)
Increase-light	Dijkstra	13,219	17.2	0	–
	A* straight-line (stale)	6,115	10.6	0	–
	A* ALT stale	3,321	12.8	0	–
	A* ALT refreshed	1,154	4.5	0	0.19
Increase-heavy	Dijkstra	13,064	16.5	0	–
	A* straight-line (stale)	8,895	14.3	0	–
	A* ALT stale	7,435	25.6	0	–
	A* ALT refreshed	1,189	4.8	0	0.24
Mixed with decreases	Dijkstra	13,146	14.8	0	–
	A* straight-line (stale)	5,614	8.2	**9 / 20**	–
	A* straight-line refreshed	10,825	15.1	0	0.03
	A* ALT stale	2,459	8.8	**18 / 20**	–
	A* ALT refreshed	1,111	4.0	0	0.24

(The 10k subgraph shows the same pattern: 4/20 and 6/20 non-optimal with stale straight-line / ALT after decreases; 0 otherwise.)

Findings 1. With increase-only changes, stale heuristics remain correct (0 non-optimal paths in every case) but lose pruning power: stale ALT degrades from 3.3k to 7.4k explored vertices (25k graph) as congestion grows, and is even slower than Dijkstra in time at heavy congestion (25.6 vs 16.5 ms) because of its per-vertex overhead. Refreshing the landmarks restores ~1.1k explored vertices for 0.2 s. 2. When any cost decreases, stale heuristics are **inadmissible**: ALT returned a non-optimal path in 18/20 queries and the straight-line heuristic in 9/20 (extra cost only ≈0.2–0.3% on average, but still not optimal). 3. Refreshing fixes correctness. For the straight-line heuristic the refreshed scale (smaller, since some roads became relatively cheap)

is weaker, so it explores more (10.8k) – the price of staying admissible. ALT refresh cost (0.2 s here) is cheap relative to a batch of queries but is a full graph recomputation (8 Dijkstra runs), so for frequently changing weights consider updating only landmarks whose trees are affected, or using Dijkstra/straight-line A* with a safe lower-bound scale. 4. Dijkstra needs no preprocessing and is always correct, which is why it is the robust baseline under dynamic costs.

Caveats: random (not spatially correlated) congestion, one subgraph per size, 20 queries, single-run Python timings.

Appendix B – Files and how to reproduce

See `README.md`. Pipeline: `preprocess.py` → `experiments.py` → `dynamic_costs.py` → `plot_graphs.py` / `plot_comparison.py`.